Prediction of Trajectory Launch Parameters from Simulated Timeseries Data*

Leon Galbas†

Rheinische Friedrich-Wilhelms-Universität Bonn

(Dated: 30.07.2026)

This work investigates whether launch conditions and environmental parameters can be inferred from simulated three-dimensional projectile trajectories. A dataset of 200000 trajectories is generated from a model incorporating gravity, quadratic drag, constant wind, and a simplified Magnus acceleration. Two regression approaches are compared: multilayer perceptrons operating on engineered trajectory descriptors and gated recurrent unit networks operating on resampled trajectory sequences. The models predict either the Cartesian initial velocity or a joint set of 11 launch and environmental parameters. In addition to complete, noise-free trajectories, the sequence models are evaluated under partial observation and additive Gaussian position noise. Engineered features recover the restricted initial-velocity target with very high accuracy, whereas GRU models are more effective for the joint inverse problem. Supplying numerically derived velocity and acceleration channels improves parameter recovery on clean trajectories, and random-crop augmentation substantially improves predictions from short trajectory prefixes. However, these derivative channels strongly amplify measurement noise, causing marked degradation even for small positional perturbations. The results demonstrate both the potential of sequence models for physics-informed parameter inference and the importance of matching preprocessing and training augmentation to the expected observation conditions.

I. INTRODUCTION

Inferring the causes of an observed trajectory is an inverse problem: instead of computing the motion from known initial and environmental conditions, one seeks to reconstruct those conditions from measured positions over time. For projectile motion, the task becomes nontrivial when drag, wind, and spin-induced Magnus forces act simultaneously, because several parameters can produce similar changes in range, curvature, or flight time. Reliable inference is additionally complicated when only an early part of the flight is visible or when measured positions contain noise.[1][2]

This project studies the problem in a controlled synthetic setting in which the underlying physical parameters are known exactly. Trajectories are simulated in three dimensions, and two machine-learning representations are compared: manually engineered scalar descriptors processed by a multilayer perceptron (MLP), and complete or partial time series processed by gated recurrent unit (GRU) networks.[3] The models are used to infer the initial velocity together with wind, spin direction, and effective drag and Magnus parameters.

The central questions are whether sequence models improve the joint recovery of physical parameters, how much performance changes when only a trajectory prefix is observed, and whether derivative features remain useful under measurement noise. Accordingly, the models are evaluated on complete trajectories, across different cropping fractions, and after adding Gaussian noise to the observed positions.

II. GENERATING THE DATA

The raw data used in this project consist of simulated three-dimensional flight trajectories of launched spherical sports objects, such as tennis balls, baseballs, basketballs, or similar projectiles. The goal of the simulation is not to model one specific ball type with maximal experimental accuracy, but rather to generate a broad and physically plausible family of trajectories that exhibit effects relevant for real ball flights: gravity, aerodynamic drag, wind, and spin-induced curvature due to the Magnus effect.[1][2]

For this purpose, $N = 200000$ trajectories were generated, each resampled to 10000 time points between launch and impact with the ground. Each trajectory is represented by the time-dependent position

$$\mathbf{r}(t) = \begin{pmatrix} x(t) \\ y(t) \\ z(t) \end{pmatrix},$$

where $z = 0$ denotes the ground plane. The resulting dataset is therefore well suited for testing whether machine learning models can infer launch conditions or trajectory characteristics from complete, noisy, or partially observed flight paths.

---

* Project work produced as part of the "Machine Learning for Quantum Scientists" graduate physics course lead by Prof. Dr. Matthias Schott

† s6legalb@uni-bonn.de

A. Physical Assumptions

The projectile is modeled as a point mass moving in three spatial dimensions. The height coordinate z is measured relative to a flat ground plane, and the integration is stopped once the projectile reaches $z = 0$ from above. The gravitational acceleration is assumed to be constant and homogeneous,

$$\mathbf{g} = \begin{pmatrix} 0 \\ 0 \\ -g \end{pmatrix}, \qquad g = 9{,}81\,\frac{\mathrm{m}}{\mathrm{s}^2}.$$

Furthermore, the wind velocity is assumed to be spatially and temporally constant during a single trajectory. The aerodynamic parameters are also kept constant along the trajectory, meaning that effects such as changing drag coefficients, turbulent flow transitions, deformation of the ball, or changing spin rate are neglected. The spin direction is represented by a fixed unit vector $\hat{\omega}$, and spin dynamics such as precession or spin decay are not modeled explicitly. These assumptions lead to a controlled but still non-trivial simulation setting: the generated trajectories remain interpretable from the perspective of classical mechanics, while also containing enough variation to form a challenging machine learning dataset.

B. Equations of Motion

The dynamical state of a projectile is given by its position $\mathbf{r}$ and velocity $\mathbf{v}$,

$$\mathbf{y}(t) = \begin{pmatrix} \mathbf{r}(t) \\ \mathbf{v}(t) \end{pmatrix} = \begin{pmatrix} x(t) \\ y(t) \\ z(t) \\ v_x(t) \\ v_y(t) \\ v_z(t) \end{pmatrix}.$$

The time evolution is determined by Newton's equation in first-order form,

$$\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}t} = \mathbf{v}, \qquad \frac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = \mathbf{a}.$$

In addition to gravity, two aerodynamic forces are included: quadratic drag and a simplified Magnus acceleration. Since aerodynamic forces depend on the velocity relative to the surrounding air, the relative velocity is defined as

$$\mathbf{q} = \mathbf{v} - \mathbf{u},$$

where $\mathbf{u}$ is the wind velocity. The acceleration model used in the simulation is

$$\frac{\mathrm{d}\mathbf{v}}{\mathrm{d}t} = \underbrace{\begin{pmatrix} 0 \\ 0 \\ -g \end{pmatrix}}_{\text{gravity}} - \underbrace{\beta_D \|\mathbf{q}\| \mathbf{q}}_{\text{drag}} + \underbrace{\beta_M \left(\hat{\omega} \times \mathbf{q}\right)}_{\text{Magnus effect}}. \tag{1}$$

Here, β_D is an effective drag parameter and β_M is an effective Magnus parameter. The drag term is opposite to the relative motion through the air and scales quadratically with the relative speed, since

$$\|\mathbf{q}\|\mathbf{q}$$

has magnitude $\|\mathbf{q}\|^2$. The Magnus term acts perpendicular to both the spin axis and the relative velocity, producing curved trajectories for spinning balls.[1][2] The parameters β_D and β_M absorb several physical constants, such as mass, radius, cross-sectional area, air density, drag coefficient, spin magnitude, and lift coefficient. This is useful because the purpose of the simulation is to generate a diverse family of trajectories rather than to separately identify every microscopic physical parameter.

The initial position is chosen as

$$\mathbf{r}_0 = \begin{pmatrix} 0 \\ 0 \\ \epsilon \end{pmatrix}, \qquad \epsilon = 10^{-6},$$

where the small offset ϵ prevents the event detection from immediately identifying the ball as being on the ground at $t = 0$. The initial velocity is parameterized using a speed v_0, a polar angle θ, and an azimuthal angle ϕ:

$$\mathbf{v}_0 = v_0 \begin{pmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{pmatrix}.$$

In this convention, $\theta = 0$ corresponds to a vertical launch and $\theta \approx \pi/2$ corresponds to an almost horizontal launch. The ordinary differential equation in Eq. (1) is integrated numerically until the first downward crossing of the ground plane,

$$z(t_{\mathrm{hit}}) = 0, \qquad \dot{z}(t_{\mathrm{hit}}) < 0.$$

After integration, each trajectory is resampled to a fixed number of time points using the normalized time coordinate

$$\tau = \frac{t}{t_{\mathrm{hit}}} \in [0, 1].$$

This ensures that all simulated examples have the same array shape, even though their physical flight times differ.

C. Launch and Environmental Parameters

Each trajectory is generated by randomly sampling a set of launch, aerodynamic, wind, and spin parameters. The sampled parameter vector is

$$\lambda = (v_0, \theta, \phi, u_x, u_y, u_z, \omega_x, \omega_y, \omega_z, \beta_D, \beta_M)\,.$$

The wind vector is first sampled in spherical coordinates using a wind speed $u = \|\mathbf{u}\|$, a polar angle θ_u, and an azimuthal angle ϕ_u, and is then converted to Cartesian components:

$$\mathbf{u} = u \begin{pmatrix} \sin\theta_u \cos\phi_u \\ \sin\theta_u \sin\phi_u \\ \cos\theta_u \end{pmatrix}.$$

Similarly, the spin direction is represented by a normalized vector

$$\hat{\omega} = \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}, \qquad \|\hat{\omega}\| = 1.$$

Only the spin direction is sampled explicitly, while the magnitude of the spin contribution is absorbed into the effective coefficient β_M. This reduces the number of independent degrees of freedom: instead of separately sampling spin magnitude, ball radius, mass, air density, and lift coefficient, the simulation uses the compact parameter β_M to control the overall strength of the Magnus effect. Analogously, β_D controls the overall strength of air resistance. This parameterization is advantageous for machine learning experiments because it keeps the input space lower-dimensional while still producing qualitatively realistic trajectory variations.

The default parameter ranges used in the dataset generation are summarized in Table I. The initial speed range $v_0 \in [0, 75\,\mathrm{m/s}]$ was chosen to include a wide spectrum of sports-ball launch speeds, from very slow throws to high-speed shots or hits. The launch direction is restricted to the upper hemisphere, $\theta \in [0, \pi/2)$, because downward initial launches are not considered in this dataset. The azimuthal angle $\phi \in [0, 2\pi)$ allows arbitrary horizontal launch directions. The wind speed range $u \in [0, 25\,\mathrm{m/s}]$ covers calm conditions as well as strong winds, while the wind direction is sampled over the full sphere. Finally, the ranges of β_D and β_M were chosen broadly enough to produce trajectories ranging from nearly ballistic motion to strongly damped and noticeably curved flights.

The angular sampling is performed such that directions are distributed uniformly on the relevant sphere or hemisphere. For example, instead of sampling the polar angle θ uniformly, the simulation samples $\cos\theta$ uniformly and then computes

$$\theta = \arccos(s),$$

Parameter	Range	Physical meaning
v_0	$[0, 75]$ m/s	Initial speed
θ	$[0, \pi/2)$	Launch polar angle
ϕ	$[0, 2\pi)$	Launch azimuthal angle
β_D	$[0.003, 0.04]$ 1/m	Effective drag strength
β_M	$[0, 0.3]$ 1/s	Effective Magnus strength
u	$[0, 25]$ m/s	Wind speed
θ_u	$[0, \pi]$	Wind polar angle
ϕ_u	$[0, 2\pi)$	Wind azimuthal angle
$\hat{\omega}$	$\|\hat{\omega}\| = 1$	Spin-axis direction

Table I. Default parameter ranges used for trajectory generation. Angles are given in radians.

where s is uniformly distributed in the corresponding interval. This avoids oversampling directions close to the polar axis, which would occur for naive uniform sampling of θ. The same idea is used for the wind direction over the full sphere. The spin-axis direction $\hat{\omega}$ is generated by drawing a three-dimensional Gaussian random vector and normalizing it to unit length, which also produces an isotropic distribution of directions.

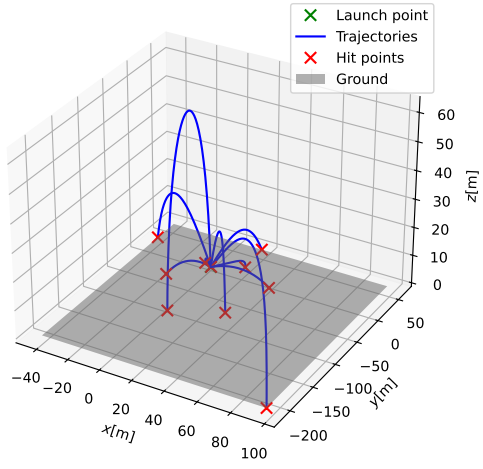


Figure 1. Example 3D trajectories generated by the simulation. The curves illustrate the diversity of flight paths caused by different launch speeds, directions, drag strengths, wind vectors, and Magnus parameters.

Overall, the simulation provides a controlled synthetic dataset with known physical parameters and tunable sources of trajectory variation. This makes it possible to systematically study how different machine learning models behave when trained on full trajectories, engineered features, noisy observations, or cropped partial trajectories.

III. METHODS

This section describes the machine learning methods used to infer physical launch and environmental parameters

from simulated projectile trajectories. Two complementary model classes are considered: a multilayer perceptron (MLP) applied to manually engineered scalar trajectory descriptors, and gated recurrent unit (GRU) networks applied directly to time-resolved trajectory data. The overall motivation is to compare a compact feature-based representation against a sequence-based representation that can, in principle, learn relevant temporal structure directly from the observed motion.

A. Feature-Based Regression with a Multilayer Perceptron

As a non-recurrent baseline, we use a multilayer perceptron that maps a fixed-dimensional vector of scalar trajectory features to the desired physical target parameters. In the implementation, the MLP first flattens the input, then applies a sequence of fully connected linear layers with ReLU activations, followed by a final linear output layer. For an input feature vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, the model can be written schematically as

$$\begin{aligned} \mathbf{h}_1 &= \sigma\left(W_1\mathbf{x} + \mathbf{b}_1\right), \\ \mathbf{h}_{\ell+1} &= \sigma\left(W_{\ell+1}\mathbf{h}_\ell + \mathbf{b}_{\ell+1}\right), \qquad \ell = 1, \ldots, L-1, \\ \hat{\mathbf{y}} &= W_{\text{out}}\mathbf{h}_L + \mathbf{b}_{\text{out}}, \end{aligned}$$

where σ denotes the ReLU[4] activation function and $\hat{\mathbf{y}}$ is the predicted target vector. The MLP experiments use scalar inputs with 24 engineered features, and the output dimension is either 3 for predicting the Cartesian initial velocity or 11 for predicting the full set of selected launch and environmental parameters.

The scalar features are designed to summarize the most important geometric and kinematic properties of the trajectory. They include the initial and final positions (x_0, y_0, z_0) and (x_T, y_T, z_T), the total flight time T, the displacement components $(\Delta x, \Delta y, \Delta z)$, the Euclidean start-to-end distance, the total arc length of the trajectory, the apex position $(x_{\text{peak}}, y_{\text{peak}}, z_{\text{peak}})$, the time of the apex t_{peak}, and finite-difference estimates of the initial and final velocity components and speeds. These quantities provide physically interpretable information about range, curvature, maximum height, flight duration, and endpoint behavior. For example, the initial finite-difference velocity estimate is computed from the first two sampled trajectory points, while the final velocity estimate is computed from the last two sampled points. The total trajectory length is computed by summing the Euclidean distances between successive sampled positions. The apex is identified as the point of maximal height z, which gives both the peak position and the corresponding time.

For some experiments, the desired target is not the spherical launch representation (v_0, θ, ϕ), but the Cartesian initial velocity (v_x, v_y, v_z). This target is computed using

$$\begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix} = v_0 \begin{pmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{pmatrix}$$

which is implemented as a preprocessing target transformation. Using Cartesian velocity components avoids angular discontinuities in ϕ and gives a target representation that is often easier for regression models to learn.

B. Sequence-Based Regression with Gated Recurrent Units

The second model class is based on gated recurrent units, which are recurrent neural networks designed to process sequential data. Instead of receiving a single engineered feature vector, the GRU receives a time series of trajectory observations such as

$$\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T, \qquad \mathbf{x}_t = \begin{pmatrix} t_t & x_t & y_t & z_t \end{pmatrix}^\top,$$

or an extended version that also contains velocity and acceleration estimates. The GRU then returns a final hidden state, which is passed through a small feed-forward prediction head consisting of a linear layer, ReLU activation, dropout, and a final linear layer. In the endpoint-augmented variants, a second prediction head is added to predict the two-dimensional hit point separately from the physical parameter vector.

A GRU updates its hidden state using gating mechanisms that control how much past information is retained and how much new information is incorporated. For an input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$, the standard GRU equations are

$$\begin{aligned} \mathbf{z}_t &= \sigma\left(W_z\mathbf{x}_t + U_z\mathbf{h}_{t-1} + \mathbf{b}_z\right), \\ \mathbf{r}_t &= \sigma\left(W_r\mathbf{x}_t + U_r\mathbf{h}_{t-1} + \mathbf{b}_r\right), \\ \tilde{\mathbf{h}}_t &= \tanh\left(W_h\mathbf{x}_t + U_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h\right), \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned}$$

Here, $\mathbf{z}_t$ is the update gate, $\mathbf{r}_t$ is the reset gate, $\tilde{\mathbf{h}}_t$ is the candidate hidden state, and $\odot$ denotes element-wise multiplication.[3] The update gate allows the network to preserve information over many time steps, while the reset gate controls how strongly the previous hidden state contributes to the candidate state. This makes GRUs suitable for time-series analysis because they learn a compressed representation of the trajectory history rather than treating each time point independently. In this project, the final hidden state $\mathbf{h}_T$ is interpreted as a learned summary of the observed flight path and is used to predict the physical parameters.

The GRU models are particularly appropriate for this problem because projectile trajectories contain tempo-

rally ordered information. Quantities such as curvature, deceleration, and asymmetry are not fully described by individual positions alone, but by their evolution over time. A recurrent architecture can use early and late parts of the trajectory jointly, and it can also be evaluated on cropped sequences, where only a prefix of the trajectory is available.[3] The training code supports random cropping of time-series batches by selecting only the first k time points, where k is sampled between fractions of the full sequence length. This allows the model to learn from partial trajectories and is useful for studying how much of the flight path is needed to infer the underlying launch and environmental parameters.

C. Data Preprocessing and Representation

The raw simulator output consists of time arrays, three-dimensional positions, launch parameters, and metadata stored in immutable raw samples. During preprocessing, each raw sample is converted into a processed representation containing trajectory channels t, x, y, z, scalar features, targets, and optionally metadata. The dataset builder applies a configurable pipeline of transformations to each raw sample and then writes the selected features and targets into PyTorch tensors. For time-series data, the resulting feature tensor has shape

$$(N, n_{\text{features}}, n_{\text{timepoints}}),$$

whereas scalar-feature data are stored as

$$(N, n_{\text{features}}).$$

The target tensor has shape (N, n_{targets}) in both cases.

A central preprocessing step for the GRU models is resampling. Although the raw trajectories were generated with many time points, the GRU experiments use a reduced fixed-length representation with 100 time points. The resampling transform first chooses new time points between the minimum and maximum time of the trajectory and then interpolates the trajectory channels using a cubic spline.[5] The implemented sampling distributions include even, logarithmic, uniform, and Gaussian sampling over the normalized time interval. In the main GRU training configurations, even resampling is used, so the trajectory is represented by uniformly spaced time samples. This reduces computational cost while preserving a standardized input length for mini-batch training.

Additional time-series features are obtained by numerical differentiation. Velocity channels are computed as

$$v_i(t) \approx \frac{\mathrm{d}r_i}{\mathrm{d}t}, \qquad i \in \{x, y, z\},$$

using numerical gradients of the position channels with respect to time. Acceleration channels are then computed analogously from the velocity channels. Some GRU configurations therefore use only (t, x, y, z), while others use the extended feature set $(t, x, y, z, v_x, v_y, v_z, a_x, a_y, a_z)$. The motivation for adding velocities and accelerations is that the forces in the physical simulation act through velocity-dependent drag and Magnus terms, so derivative information may make the underlying parameters easier to identify. At the same time, numerical differentiation can amplify noise, so the benefit of these features must be evaluated empirically. [6]

All models use standardization of features and targets. Standardization transforms each feature to have zero mean and unit variance, ensuring that all input features are on a comparable scale. This improves the stability and efficiency of gradient-based optimization, allowing neural networks to converge faster and preventing features with larger numerical values from disproportionately influencing the learning process.[7] For scalar inputs, feature means and standard deviations are computed over the training samples, while for time-series inputs they are computed over both the sample and time dimensions for each feature channel. Targets are standardized using the training-set mean and standard deviation for each output dimension. The standardized quantities are therefore

$$\tilde{x} = \frac{x - \mu_x}{\sigma_x + \epsilon}, \qquad \tilde{y} = \frac{y - \mu_y}{\sigma_y + \epsilon},$$

with a small numerical constant $\epsilon = 10^{-8}$ used for stability. The same scaling factors are then applied to the corresponding test data, which avoids information leakage from the test set into the training process. For later standalone evaluation runs, the saved feature and target scaling factors can be loaded and reused.

IV. EXPERIMENTAL SETUP

A. Dataset Split and Preprocessing

All experiments use the same repository of $N = 200000$ simulated trajectories described in Section II. A pseudo-random permutation divides the data into 160000 training trajectories and 40000 test trajectories. The latter split is used to control the learning-rate scheduler, select the best checkpoint, and report the final metrics.

The scalar features and time-series channels are standardized as described in Section III. Only statistics of the training split are used, and the same saved statistics are applied during all later evaluations. For the GRU models, each raw trajectory is represented by 100 evenly spaced samples obtained by cubic-spline interpolation. The noise-free trajectory is used to construct all targets, including the impact point in the endpoint-augmented models.

B. Model Configurations

Table II summarizes the eight configurations. The two MLPs receive the 24 engineered features introduced in Section III. The first MLP predicts only the Cartesian initial velocity, whereas the second predicts all 11 launch and environmental parameters. The GRU prediction head maps the final hidden state through a 128-unit linear layer, a ReLU activation, dropout with probability 0.1, and a final linear output layer. Two-layer GRUs additionally use dropout 0.1 between recurrent layers. Configurations denoted by "derivatives" receive the ten channels

$$(t, x, y, z, v_x, v_y, v_z, a_x, a_y, a_z),$$

whereas the remaining GRUs receive only (t, x, y, z).

During crop-augmented training, a single prefix length is sampled uniformly between 20 and 100 time points for each mini-batch, and the same length is used for every trajectory in that batch. For configurations 07 and 08, the test loss used during training is evaluated at $f = 0.5$. Their two heads minimize

$$\mathcal{L} = \mathrm{MSE}\left(\hat{\lambda}, \lambda\right) + 0.5\,\mathrm{MSE}\left(\hat{\mathbf{r}}_{\mathrm{hit}}, \mathbf{r}_{\mathrm{hit}}\right), \tag{2}$$

where both target blocks are standardized separately before the loss is computed.

C. Optimization and Model Selection

All models are optimized with Adam using a batch size of 256, an initial learning rate of 10^{-3}, and mean squared error in standardized target space.[8] A `ReduceLROnPlateau` scheduler multiplies the learning rate by 0.1 after 10 epochs without improvement, down to a minimum of 10^{-6}. Early stopping is configured with a tolerance of 32 non-improving epochs. The checkpoint with the lowest test loss is retained and used in every subsequent evaluation.[9]

The data permutation is reproducible because its seed is set explicitly. However, due to processing power constraints, only one trained instance of each configuration is available.The reported differences therefore describe these particular training runs. Uncertainty across initializations is not quantified.

The whole machine learning framework is implemented using PyTorch.[10] Additional numerical calculations are performed using the NumPy and SciPy libraries.[11][12] Visualizations and plots are realized in Matplotlib.[13]

V. EVALUATION

A. Regression Metrics

Metrics are computed after undoing target standardization, separately for every physical output. For target j, with residual $e_{ij} = \hat{y}_{ij} - y_{ij}$ on n test samples, the principal metrics are

$$\mathrm{RMSE}_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_{ij}^2}, \tag{3}$$

$$\mathrm{MAE}_j = \frac{1}{n}\sum_{i=1}^{n} |e_{ij}|, \tag{4}$$

$$R_j^2 = 1 - \frac{\sum_i e_{ij}^2}{\sum_i \left(y_{ij} - \bar{y}_j\right)^2}. \tag{5}$$

RMSE retains the unit of the corresponding target and penalizes large errors more strongly than MAE, while R^2 is dimensionless. Median and maximum absolute error, explained variance, mean bias, residual standard deviation, Pearson correlation, MAPE, and sMAPE are also recorded. Percentage errors are not used as primary evidence because several Cartesian and spin-direction components can be close to zero, for which relative errors become unstable.[14]

The supplied sweep plots flatten all output components before calculating an "overall RMSE". Since the 11 targets include velocities, dimensionless spin components, β_D in 1/m, and β_M in 1/s, this aggregate is not a dimensionally meaningful physical error. Figures 3 and 4 are therefore used only to visualize qualitative model trends. Quantitative comparisons are based on per-target metrics.

B. Robustness Protocol

The cropping sweep evaluates all six GRUs on deterministic trajectory prefixes

$$f \in \{0.1, 0.2, \ldots, 1.0\}.$$

For a sequence of length $T = 100$, the model receives the first $\max(2, \lfloor fT \rfloor)$ samples. The sweep uses the same 40000 test trajectories and targets at every fraction. It therefore isolates sensitivity to the amount of observed flight history.

For the noise sweep, independent zero-mean Gaussian perturbations are added to every resampled position coordinate and time point,

$$\tilde{\mathbf{r}}_k = \mathbf{r}_k + \epsilon_k, \qquad \epsilon_k \sim \mathcal{N}\left(\mathbf{0}, \sigma^2 I_3\right), \tag{6}$$

ID	Model	Input	Architecture	Output	Crop augmentation
01	MLP velocity	24 scalar features	$64, 32$	v_x, v_y, v_z	no
02	MLP all parameters	24 scalar features	$128, 64, 32$	11 parameters	no
03	GRU positions	4 channels	1 layer, $h = 128$	11 parameters	no
04	GRU derivatives	10 channels	2 layers, $h = 128$	11 parameters	no
05	GRU cropped	4 channels	1 layer, $h = 128$	11 parameters	$f \in [0.2, 1.0]$
06	GRU cropped derivatives	10 channels	2 layers, $h = 128$	11 parameters	$f \in [0.2, 1.0]$
07	GRU cropped + endpoint	4 channels	1 layer, $h = 128$	11 parameters + 2D endpoint	$f \in [0.2, 1.0]$
08	GRU cropped derivatives + endpoint	10 channels	2 layers, $h = 128$	11 parameters + 2D endpoint	$f \in [0.2, 1.0]$

Table II. Model configurations used in the experiments. The symbol f denotes the observed fraction of a trajectory.

with

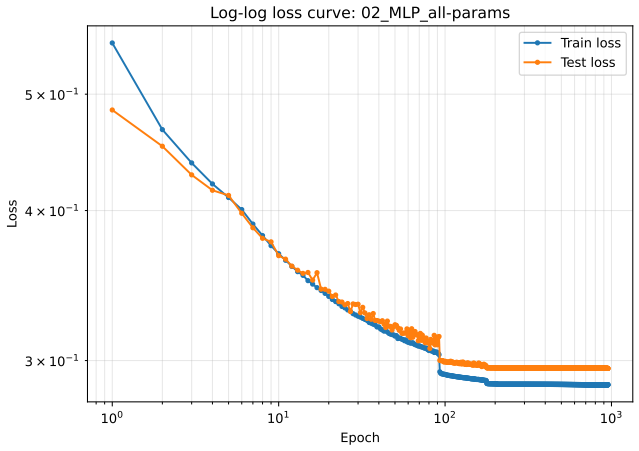
$$\sigma \in (0,\ 0{,}01,\ 0{,}02,\ 0{,}05,\ 0{,}10,\ 0{,}20,\ 0{,}50 \text{ and } 1{,}00)\,\text{m}.$$

The noise is applied after resampling but before velocity and acceleration channels are computed and before saved clean-data standardization factors are applied. Thus, σ is expressed in metres, and derivative-based models also receive derivatives of the noisy positions.[6] The physical trajectories themselves are not re-simulated: the preprocessing pipeline is rebuilt from the same raw repository, and the same split indices are reproduced.

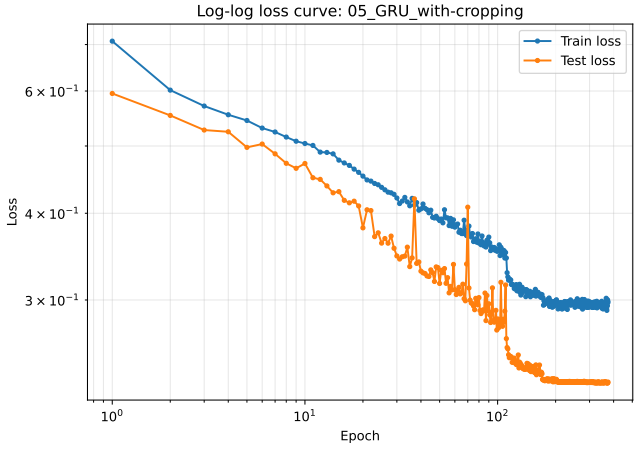
VI. RESULTS

A. Training Behaviour

The training and test loss curves decrease for all eight configurations, as illustrated schematically in Figure 2. The curves labelled "test" in the generated plots correspond to the test model-selection split defined above. The selected checkpoint occurs at epoch 953 for the velocity-only MLP and at epoch 947 for the all-parameter MLP. For the GRUs, the selected epochs are 413, 159, 343, 251, 399, and 390 for configurations 03–08, respectively. Because the objectives contain different numbers and types of standardized outputs, especially for the endpoint models in Eq. (2), absolute loss values should not be compared directly across every configuration.



(a)



(b)

Figure 2. Training and test loss as a function of epoch. The log-log representation makes both the rapid initial decrease and later convergence visible. Further loss curves can be found in the appendix (see Sec. VII).

B. Performance on Complete, Noise-Free Trajectories

The velocity-only MLP achieves RMSE values of 0,006 40 m/s, 0,006 77 m/s, and 0,003 04 m/s for v_x, v_y, and v_z, respectively. This near-exact reconstruction is expected because its engineered inputs include a finite-difference estimate of the initial velocity. It should therefore be interpreted as a task-specific feature-engineering baseline rather than as evidence that the remaining physical parameters are equally identifiable.

Table III compares representative outputs for the all-parameter models. The MLP remains competitive for launch velocity, but the GRUs recover wind, spin direction, drag, and Magnus strength more accurately. Adding velocity and acceleration channels to the full-trajectory GRU reduces the RMSE of u_x from 2,67 m/s to 1,36 m/s, of β_D from 0,003 00/m to 0,001 27/m, and of β_M from 0,0378/s to 0,0190/s. The corresponding R^2 values increase from 0.899 to 0.974, from 0.920 to 0.986,

and from 0.810 to 0.952, respectively.

Target	MLP all	GRU positions	GRU derivatives	GRU cropped derivatives
v_x [m/s]	1.016 (0.998)	1.793 (0.995)	1.027 (0.998)	1.223 (0.998)
u_x [m/s]	3.857 (0.789)	2.667 (0.899)	1.359 (0.974)	1.413 (0.972)
ω_x	0.413 (0.488)	0.247 (0.817)	0.129 (0.950)	0.133 (0.947)
β_D [1/m]	0.00445 (0.825)	0.00300 (0.920)	0.00127 (0.986)	0.00121 (0.987)
β_M [1/s]	0.0664 (0.413)	0.0378 (0.810)	0.0190 (0.952)	0.0191 (0.952)

Table III. Per-target test performance on complete, noise-free trajectories. Each entry reports RMSE followed by R^2 in parentheses. Model names correspond to configurations 02, 03, 04, and 06 in Table II.

Random-crop augmentation slightly sacrifices complete-trajectory accuracy for some outputs, but configuration 06 remains close to configuration 04. The endpoint-augmented derivative model likewise gives comparable parameter errors, for example RMSE(β_M) = 0,0184/s. Its endpoint MAEs are 1,22 m in x and 1,17 m in y, while the corresponding RMSEs are 4,65 m and 3,35 m. The gap between MAE and RMSE indicates that a minority of comparatively large endpoint errors remains. The auxiliary endpoint objective does not yield a consistent improvement of the 11 physical-parameter predictions over configuration 06.

C. Dependence on the Observed Trajectory Fraction

Figure 3 shows that GRUs trained only on complete trajectories deteriorate strongly when only an early prefix is available. Crop augmentation substantially reduces this distribution shift. For configuration 06 at $f = 0.5$, the RMSEs are 1,08 m/s for v_x, 1,64 m/s for u_x, 0.150 for ω_x, 0,001 36/m for β_D, and 0,0223/s for β_M. These values are close to the corresponding full-trajectory errors in Table III. Even at $f = 0.1$, configuration 06 yields RMSE(v_x) = 2,16 m/s, compared with 6,62 m/s for the derivative GRU trained only on complete trajectories. The remaining deterioration of wind, spin, and aerodynamic parameters at very small f is physically plausible because these quantities influence curvature and deceleration over time, so their effects are only weakly expressed at the beginning of a flight.

D. Robustness to Gaussian Position Noise

All GRUs are sensitive to measurement noise, and the degradation is already pronounced at σ = 0,01 m, as shown in Figure 4. For the derivative GRU without crop augmentation, the RMSE of v_x increases from 1,03 m/s to 21,0 m/s, while the RMSE of u_x increases from 1,36 m/s to 9,50 m/s. Over the same change, the β_D error increases from 0,001 27/m to 0,0120/m. This behaviour is consistent with the fact that velocity and especially acceleration are obtained by differentiating already perturbed positions.

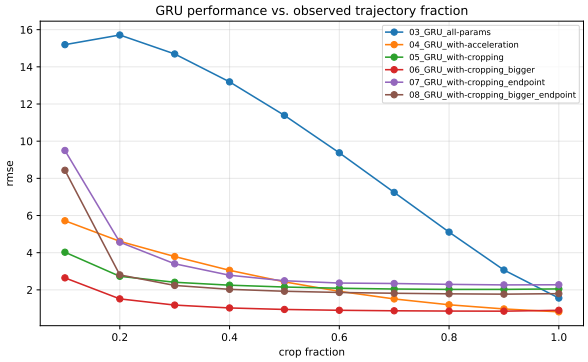


Figure 3. Diagnostic aggregate RMSE of the GRU configurations as a function of the observed trajectory fraction. Since the aggregate mixes outputs with different physical units, per-target values are used for quantitative interpretation.

No single configuration dominates for every noisy target. At σ = 0,01 m, the position-only endpoint model has a lower v_x RMSE of 6,94 m/s, but its u_x RMSE is 20,3 m/s, compared with 9,50 m/s for the derivative GRU. The results therefore reveal a trade-off between clean-data accuracy and target-dependent robustness. They also show that crop augmentation alone is not a substitute for noise augmentation: none of the training configurations adds Gaussian observation noise. A natural extension is to train with noise levels representative of the intended measurement system [15], or to estimate derivatives using a smoothing or regularized procedure [6].

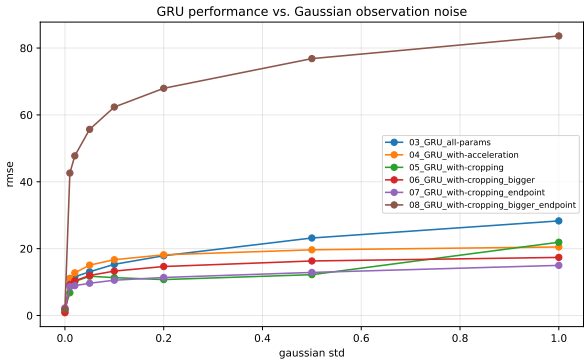


Figure 4. Diagnostic aggregate RMSE of the GRU configurations under independent Gaussian position noise. Noise is added before derivative features are computed. The aggregate curve is used qualitatively because it combines targets with different units.

Overall, engineered velocity features solve the restricted initial-velocity task extremely accurately, whereas sequence models are more effective for the joint inverse problem. For complete, clean trajectories, explicitly supplied derivatives provide the strongest overall parameter recovery. For partial trajectories, random-crop training is essential and retains useful accuracy down to substantially shorter prefixes. In contrast, the noise experiment

exposes a major limitation of the present preprocessing pipeline: numerical derivatives improve clean-data accuracy but amplify positional noise.

VII. CONCLUSION

This project compared feature-based MLPs and sequence-based GRUs for inferring physical parameters from simulated three-dimensional projectile trajectories. The restricted initial-velocity task was solved extremely accurately by an MLP using engineered features, with test RMSEs of 0,006 40 m/s, 0,006 77 m/s, and 0,003 04 m/s for v_x, v_y, and v_z, respectively. This result is primarily attributable to the inclusion of a finite-difference estimate of the initial velocity and therefore demonstrates the value of task-specific feature engineering. For the more demanding joint prediction of 11 launch and environmental parameters, the GRU models generally outperformed the all-parameter MLP, particularly for wind, spin direction, drag, and Magnus strength.

On complete, noise-free trajectories, explicitly providing velocity and acceleration channels produced the most accurate overall parameter recovery. For example, relative to the position-only full-trajectory GRU, the derivative GRU reduced the test RMSE of u_x from 2,67 m/s to 1,36 m/s and that of β_D from 0,003 00/m to 0,001 27/m. Random-crop augmentation was essential when only a prefix of the flight was available. The crop-trained derivative model retained useful performance at short observation fractions and, at $f = 0.1$, reduced the v_x RMSE from 6,62 m/s for the model trained only on complete trajectories to 2,16 m/s. The auxiliary impact-point objective did not consistently improve recovery of the physical parameters.

The Gaussian-noise experiments exposed the main weakness of the present approach. Although numerical derivatives are highly informative for clean inputs, differentiating perturbed positions amplifies measurement noise. For the derivative GRU, adding only $\sigma = 0{,}01$ m of positional noise increased the v_x RMSE from 1,03 m/s to 21,0 m/s. Crop augmentation did not prevent this degradation because none of the models was trained with noisy observations. Future work should therefore include noise augmentation matched to the measurement process and investigate smoothing, regularized differentiation, or architectures that infer kinematic information directly from positions.[6]

The conclusions are limited to the simulated parameter ranges and physical assumptions used here. In addition, the same test split was used for checkpoint selection and final evaluation, only one training run was available for each configuration, and no independent simulated or experimental test set was used. Repeated training with controlled random seeds, a separate final test set, and validation on real trajectory measurements would be required for stronger claims about generalization and practical applicability. Within these limitations, the experiments show that GRUs can recover several latent physical parameters from trajectory data, provided that the training distribution reflects the expected degree of trajectory incompleteness and measurement noise.

DECLARATION ON THE USE OF AI-ASSISTED TOOLS

During the preparation of this paper, ChatGPT 5.5 Full was used for spell checking, grammar checking, and general language improvement.

The source code associated with this work was written predominantly by hand. In cases where a file was generated with the assistance of an AI tool, this is explicitly disclosed in a comment at the beginning of the respective file in the accompanying GitHub repository.

The author reviewed and verified all AI-assisted revisions and remains fully responsible for the content of the paper and the associated source code.

DATA AVAILABILITY

The full source code used for simulating the trajectories and training/evaluating the machine learning models in the project is available on GitHub under `https://github.com/leon-galbas/physics7516-ml-final-project`.

[1] R. D. Mehta, Aerodynamics of sports balls, Annual Review of Fluid Mechanics **17**, 151 (1985).

[2] J. E. Goff and M. J. Carré, Trajectory analysis of a soccer ball, American Journal of Physics **77**, 1020 (2009).

[3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, Learning phrase representations using RNN encoder–decoder for statistical machine translation, in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, edited by A. Moschitti, B. Pang, and W. Daelemans (Association for Computational Linguistics, Doha, Qatar, 2014) pp. 1724–1734.

[4] V. Nair and G. E. Hinton, Rectified linear units improve restricted boltzmann machines, in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ICML'10 (Omnipress, Madison, WI, USA, 2010) p. 807–814.

[5] C. De Boor and C. De Boor, *A practical guide to splines*, Vol. 27 (springer New York, 1978).

[6] R. Chartrand, Numerical differentiation of noisy, nonsmooth data, International Scholarly Research Notices **2011**, 164564 (2011), https://onlinelibrary.wiley.com/doi/pdf/10.5402/2011/164564.
[7] Y. A. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, Efficient backprop, in *Neural Networks: Tricks of the Trade: Second Edition*, edited by G. Montavon, G. B. Orr, and K.-R. Müller (Springer Berlin Heidelberg, Berlin, Heidelberg, 2012) pp. 9–48.
[8] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization (2017), arXiv:1412.6980 [cs.LG].
[9] L. Bottou, F. E. Curtis, and J. Nocedal, Optimization methods for large-scale machine learning, SIAM Review **60**, 223 (2018), https://doi.org/10.1137/16M1080173.
[10] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, Pytorch: An imperative style, high-performance deep learning library, Advances in neural information processing systems **32** (2019).
[11] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, Array programming with NumPy, Nature **585**, 357 (2020).
[12] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python, Nature Methods **17**, 261 (2020).
[13] J. D. Hunter, Matplotlib: A 2d graphics environment, Computing in Science & Engineering **9**, 90 (2007).
[14] D. Chicco, M. J. Warrens, and G. Jurman, The coefficient of determination r-squared is more informative than smape, mae, mape, mse and rmse in regression analysis evaluation, Peerj computer science **7**, e623 (2021).
[15] C. M. Bishop, Training with noise is equivalent to tikhonov regularization, Neural Computation **7**, 108 (1995), https://direct.mit.edu/neco/article-pdf/7/1/108/812990/neco.1995.7.1.108.pdf.

APPENDIX

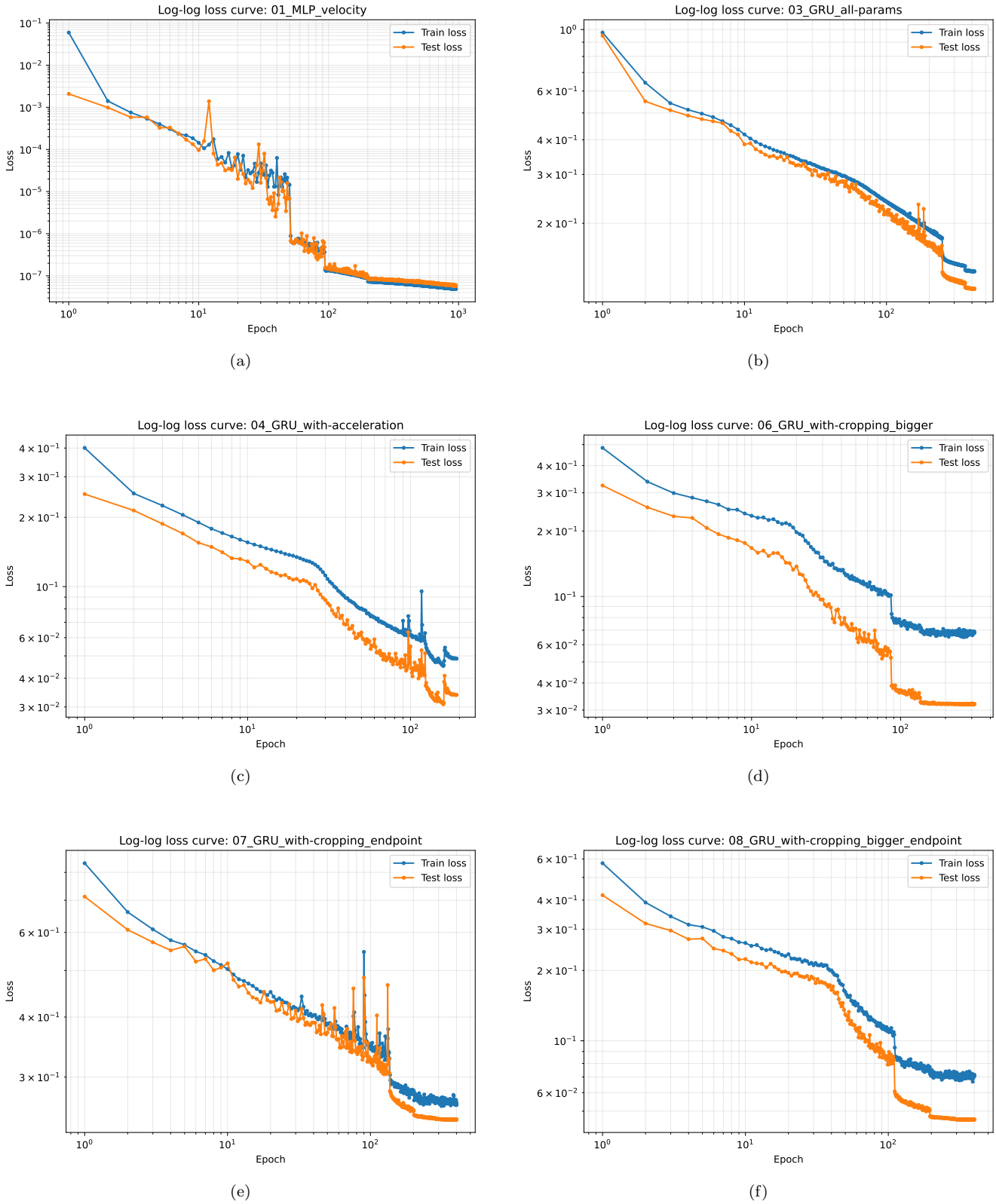


Figure 5. Additional training run loss curves.